

# ***Operating System (ISE3137)***

**Jump Together, Fly Farther!**



인하대학교 국제학부

## ***Week 6 Lab***

**ISE Department  
Prof. Mehdi Pirahandeh**

# Examples of simple commands

---

- The date command is used to display the current date and time. It can also be used by the
- superuser to set the system clock. An argument that begins with a plus sign (+) specifies a
- format string for the date command.

# Examples of simple commands

- The date command is used to display the current date and time. It can also be used by the
- superuser to set the system clock. An argument that begins with a plus sign (+) specifies a
- format string for the date command.

```
ubuntu@instance-20230321-1112:~$ date
Wed Apr  5 06:52:20 UTC 2023
ubuntu@instance-20230321-1112:~$ date +%R
06:53
ubuntu@instance-20230321-1112:~$ date +"%m-%d-%y"
04-05-23
ubuntu@instance-20230321-1112:~$ date +"%D"
04/05/23
ubuntu@instance-20230321-1112:~$ date +"%T"
07:14:10
ubuntu@instance-20230321-1112:~$
```

# Examples of simple commands

---

- **Get Parse Current Date:**
- You can get the current system date and time value using ``date`` command. Every part of date and time value can be parsed using `'Y'`, `'m'`, `'d'`, `'H'`, `'M'` and `'S'`. Create a new file named `'date_parse.sh'` and add the following code to separate day, month, year, hour, minute and second values.

# Examples of simple commands

- **Get Parse Current Date:**

- *#!/bin/bash*  
Year=`date +%Y`  
Month=`date +%m`  
Day=`date +%d`  
Hour=`date +%H`  
Minute=`date +%M`  
Second=`date +%S`  
echo `date`  
echo "Current Date is: \$Day-\$Month-\$Year"  
echo "Current Time is: \$Hour:\$Minute:\$Second"

- Run the file with bash command.
- **\$ bash date\_parse.sh**

# Examples of simple commands

- **Get Parse Current Date:**

- *#!/bin/bash*  
Year=`date +%Y`  
Month=`date +%m`  
Day=`date +%d`  
Hour=`date +%H`  
Minute=`date +%M`  
Second=`date +%S`  
echo `date`  
echo "Current Date is: \$Day-\$Month-\$Year"  
echo "Current Time is: \$Hour:\$Minute:\$Second"

- Run the file with bash command.
- **\$ bash date\_parse.sh**

# Examples of simple commands

- **Get Parse Current Date:**

```
• #!/bin/bash  
Year=`date +%Y`  
Month=`date +%m`  
Day=`date +%d`  
Hour=`date +%H`  
Minute=`date +%M`  
Second=`date +%S`  
echo `date`  
echo "Current Date is: $Day-$Month-$Year"  
echo "Current Time is: $Hour:$Minute:$Second"
```

- Finish your session with the **bash** shell.
- Use either **exit** or the **Ctrl+d** key combination to close the shell.

# Redirecting Output to a File or Program

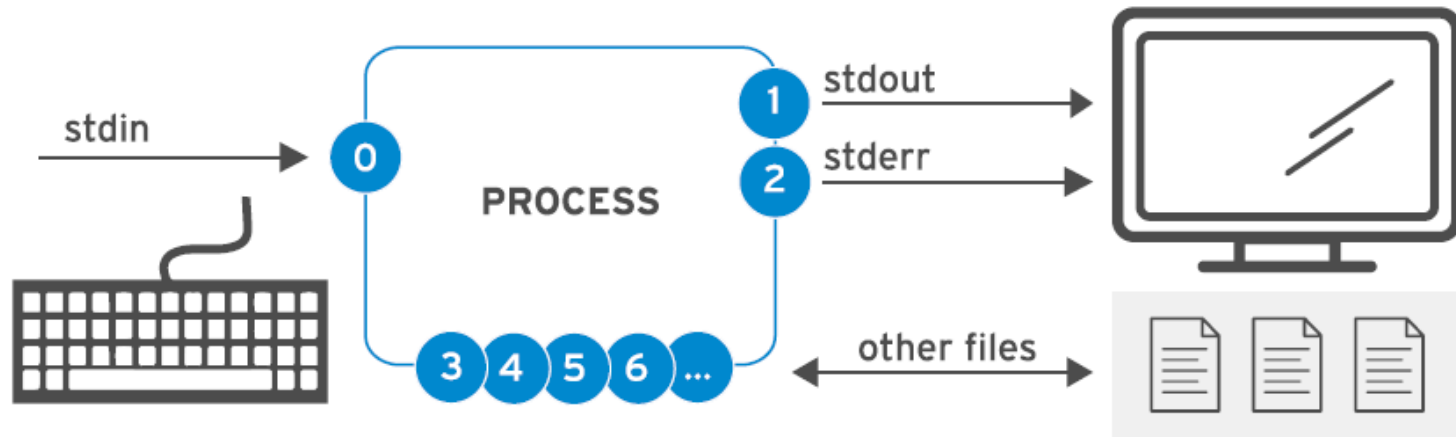


Figure 4.1: Process I/O channels (file descriptors)

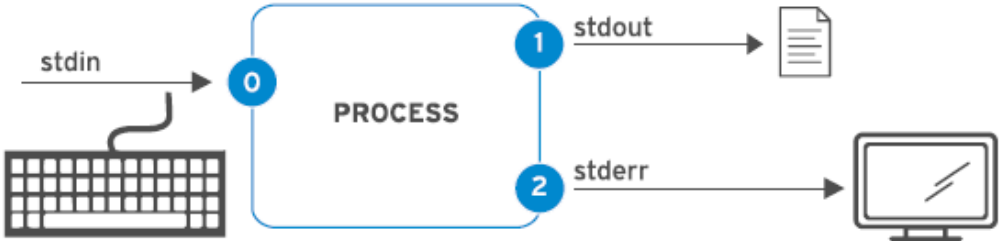
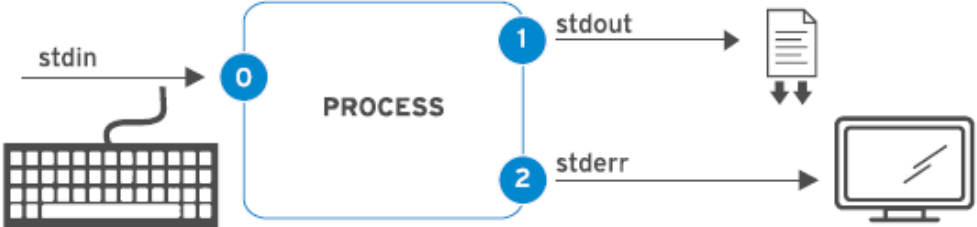
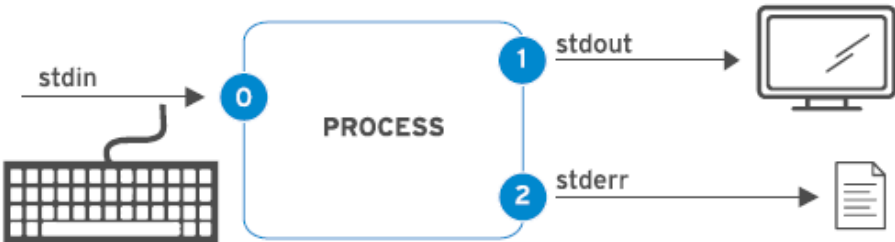
## Channels (File Descriptors)

| Number | Channel name           | Description     | Default connection | Usage             |
|--------|------------------------|-----------------|--------------------|-------------------|
| 0      | <b>stdin</b>           | Standard input  | Keyboard           | read only         |
| 1      | <b>stdout</b>          | Standard output | Terminal           | write only        |
| 2      | <b>stderr</b>          | Standard error  | Terminal           | write only        |
| 3+     | <b><i>filename</i></b> | Other files     | <i>none</i>        | read and/or write |

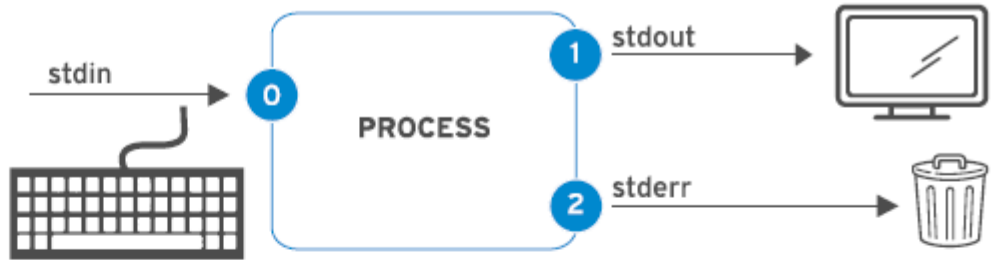
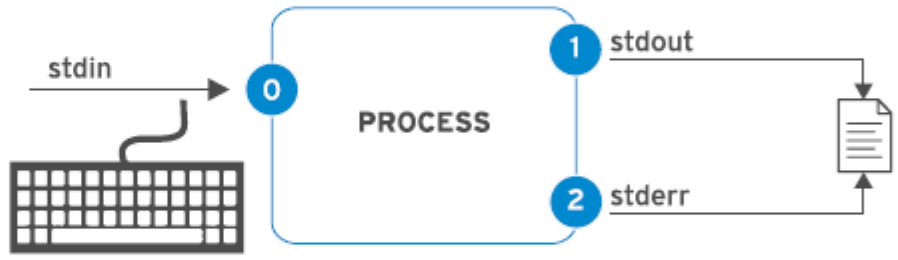
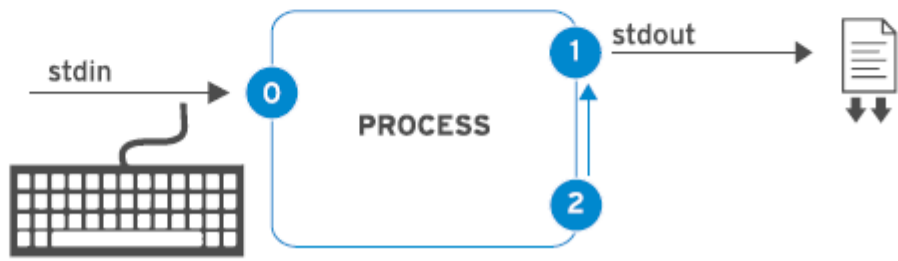


# Redirecting Output to a File or Program

## Output Redirection Operators

| Usage                     | Explanation                                | Visual aid                                                                          |
|---------------------------|--------------------------------------------|-------------------------------------------------------------------------------------|
| <code>&gt;file</code>     | redirect <b>stdout</b> to overwrite a file |   |
| <code>&gt;&gt;file</code> | redirect <b>stdout</b> to append to a file |   |
| <code>2&gt;file</code>    | redirect <b>stderr</b> to overwrite a file |  |

# Redirecting Output to a File or Program

|                                       |                                                                         |                                                                                     |
|---------------------------------------|-------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <code>2&gt;/dev/null</code>           | discard <b>stderr</b> error messages by redirecting to <b>/dev/null</b> |   |
| <code>&gt;file 2&gt;&amp;1</code>     | redirect <b>stdout</b> and <b>stderr</b> to overwrite the same file     |   |
| <code>&amp;&gt;file</code>            |                                                                         |                                                                                     |
| <code>&gt;&gt;file 2&gt;&amp;1</code> | redirect <b>stdout</b> and <b>stderr</b> to append to the same file     |  |
| <code>&amp;&gt;&gt;file</code>        |                                                                         |                                                                                     |

# Redirecting Output to a File or Program



- Save a timestamp for later reference.

```
[student@desktopX ~]$ date > /tmp/saved-timestamp
```

- Copy the last 100 lines from a log file to another file.

```
[student@desktopX ~]$ tail -n 100 /var/log/dmesg > /tmp/last-100-boot-messages
```

- Concatenate four files into one.

```
[student@desktopX ~]$ cat file1 file2 file3 file4 > /tmp/all-four-in-one
```

- List the home directory's hidden and regular file names into a file.

```
[student@desktopX ~]$ ls -a > /tmp/my-file-names
```

- Append output to an existing file.

# Redirecting Output to a File or Program

```
[student@desktopX ~]$ echo "new line of information" >> /tmp/many-lines-of-information  
[student@desktopX ~]$ diff previous-file current-file >> /tmp/tracking-changes-made
```

- In the next examples, errors are generated since normal users are denied access to system directories. Redirect errors to a file while viewing normal command output on the terminal.

```
[student@desktopX ~]$ find /etc -name passwd 2> /tmp/errors
```

- Save process output and error messages to separate files.

```
[student@desktopX ~]$ find /etc -name passwd > /tmp/output 2> /tmp/errors
```

- Ignore and discard error messages.

```
[student@desktopX ~]$ find /etc -name passwd > /tmp/output 2> /dev/null
```

- Store output and generated errors together.

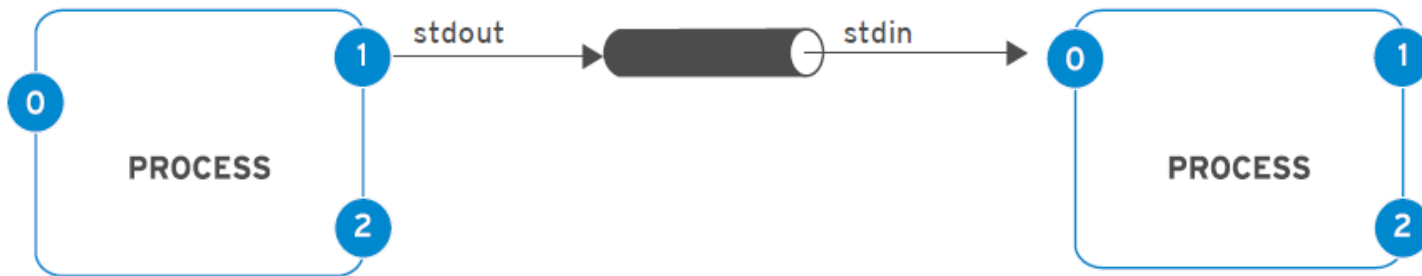
```
[student@desktopX ~]$ find /etc -name passwd &> /tmp/save-both
```

- Append output and generated errors to an existing file.

```
[student@desktopX ~]$ find /etc -name passwd >> /tmp/save-both 2>&1
```

# Redirecting Output to a File or Program

A *pipeline* is a sequence of one or more commands separated by `|`, the *pipe* character. A pipe connects the standard output of the first command to the standard input of the next command.



*Figure 4.8: Process I/O piping*

Pipelines allow the output of a process to be manipulated and formatted by other processes before it is output to the terminal. One useful mental image is to imagine that data is "flowing" through the pipeline from one process to another, being altered in slight ways by each command in the pipeline through which it passes.

# Redirecting Output to a File or Program



## Pipeline examples

This example takes the output of the **ls** command and uses **less** to display it on the terminal one screen at a time.

```
[student@desktopX ~]$ ls -l /usr/bin | less
```

The output of the **ls** command is piped to **wc -l**, which counts the number of lines received from **ls** and prints that to the terminal.

```
[student@desktopX ~]$ ls | wc -l
```

In this pipeline, **head** will output the first 10 lines of output from **ls -t**, with the final result redirected to a file.

```
[student@desktopX ~]$ ls -t | head -n 10 > /tmp/ten-last-changed-files
```

## Pipelines, redirection, and tee

When redirection is combined with a pipeline, the shell first sets up the entire pipeline, then it redirects input/output. What this means is that if output redirection is used in the *middle* of a pipeline, the output will go to the file and not to the next command in the pipeline.

In this example, the output of the **ls** command will go to the file, and **less** will display nothing on the terminal.

```
[student@desktopX ~]$ ls > /tmp/saved-output | less
```

# Redirecting Output to a File or Program

The **tee** command is used to work around this. In a pipeline, **tee** will copy its standard input to its standard output and will also redirect its standard output to the files named as arguments to the command. If data is imagined to be like water flowing through a pipeline, **tee** can be visualized as a "T" joint in the pipe which directs output in two directions.

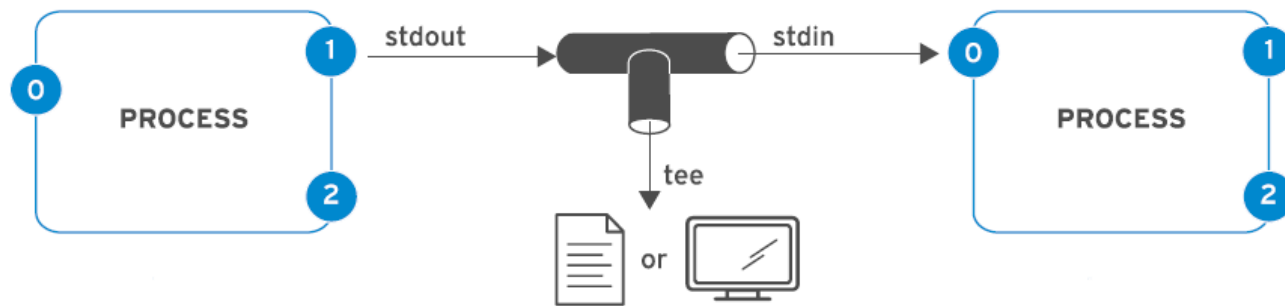


Figure 4.9: Process I/O piping with tee

## Pipeline examples using the **tee** command

This example will redirect the output of the **ls** command to the file and will pass it to **less** to be displayed on the terminal a screen at a time.

```
[student@desktopX ~]$ ls -l | tee /tmp/saved-output | less
```

If **tee** is used at the end of a pipeline, then the final output of a command can be saved and output to the terminal at the same time.

```
[student@desktopX ~]$ ls -t | head -n 10 | tee /tmp/ten-last-changed-files
```

# Users and Groups

---

What is a user?

Every process (running program) on the system runs as a particular user. Every file is owned by a particular user. Access to files and directories are restricted by user. The user associated with a running process determines the files and directories accessible to that process.

The **id** command is used to show information about the current logged-in user. Basic information about another user can also be requested by passing in the username of that user as the first argument to the **id** command.



# Users and Groups

```
[student@desktopX ~]$ id
uid=1000(student) gid=1000(student) groups=1000(student),10(wheel)
context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

To view the user associated with a file or directory, use the **ls -l** command. The third column shows the username:

```
[student@serverX ~]$ ls -l /tmp
drwx----- . 2 gdm      gdm      4096 Jan 24 13:05 orbit-gdm
drwx----- . 2 student student 4096 Jan 25 20:40 orbit-student
-rw-r--r-- . 1 root      root     23574 Jan 24 13:05 postconf
```

# Users and Groups

To view process information, use the **ps** command. The default is to show only processes in the current shell. Add the **a** option to view all processes with a terminal. To view the user associated with a process, include the **u** option. The first column shows the username:

```
[student@serverX ~]$ ps au
```

| USER    | PID  | %CPU | %MEM | VSZ    | RSS   | TTY   | STAT | START | TIME | COMMAND       |
|---------|------|------|------|--------|-------|-------|------|-------|------|---------------|
| root    | 428  | 0.0  | 0.7  | 152768 | 14400 | tty1  | Ss+  | Feb03 | 0:04 | /usr/bin/Xorg |
| root    | 511  | 0.0  | 0.0  | 110012 | 812   | ttyS0 | Ss+  | Feb03 | 0:00 | /sbin/agetty  |
| root    | 1805 | 0.0  | 0.1  | 116040 | 2580  | pts/0 | Ss   | Feb03 | 0:00 | -bash         |
| root    | 2109 | 0.0  | 0.1  | 178468 | 2200  | pts/0 | S    | Feb03 | 0:00 | su - student  |
| student | 2110 | 0.0  | 0.1  | 116168 | 2864  | pts/0 | S    | Feb03 | 0:00 | -bash         |
| student | 3690 | 0.0  | 0.0  | 123368 | 1300  | pts/0 | R+   | 11:42 | 0:00 | ps au         |

## What is a group?

Like users, groups have a name and a number (GID). Local groups are defined in **/etc/group**.

## Primary groups

- Every user has exactly one *primary group*.
- For local users, the primary group is defined by the GID number of the group listed in the fourth field of **/etc/passwd**.
- Normally, the primary group owns new files created by the user.
- Normally, the primary group of a newly created user is a newly created group with the same name as the user. The user is the only member of this *User Private Group* (UPG).

## Switching users with su

The **su** command allows a user to switch to a different user account. If a username is not specified, the *root* account is implied. When invoked as a regular user, a prompt will display asking for the password of the account you are switching to; when invoked as *root*, there is no need to enter the account password.

**su [-] <username>**

```
[student@desktopX ~]$ su -  
Password: redhat  
[root@desktopX ~]#
```

The command **su username** starts a *non-login shell*, while the command **su - username** starts a *login* shell. The main distinction is **su -** sets up the shell environment as if this were a clean login as that user, while **su** just starts a shell as that user with the current environment settings.

## Changing file/directory permissions

The command used to change permissions from the command line is **chmod**, short for "change mode" (permissions are also called the *mode* of a file). The **chmod** command takes a permission instruction followed by a list of files or directories to change. The permission instruction can be issued either symbolically (the symbolic method) or numerically (the numeric method).

### Symbolic method keywords:

```
chmod WhoWhatWhich file|directory
```

- *Who* is u, g, o, a (for user, group, other, all)
- *What* is +, -, = (for add, remove, set exactly)
- *Which* is r, w, x (for read, write, execute)

The *symbolic* method of changing file permissions uses letters to represent the different groups of permissions: **u** for user, **g** for group, **o** for other, and **a** for all.

# Users and Groups

## Numeric method:

```
chmod ### file|directory
```

- Each digit represents an access level: user, group, other.
- # is sum of r=4, w=2, and x=1.

Using the *numeric* method, permissions are represented by a three-digit (or four, when setting advanced permissions) *octal* number. A single octal digit can represent the numbers **0-7**, exactly the number of possibilities for a three-bit number.

To convert between symbolic and numeric representation of permissions, we need to know how the mapping is done. In the three-digit octal (numeric) representation, each digit stands for one group of permissions, from left to right: user, group, and other. In each of these groups, start with **0**. If the read permission is present, add **4**. Add **2** if write is present, and **1** for execute.

Numeric permissions are often used by advanced administrators since they are shorter to type and pronounce, while still giving full control over all permissions.

# Users and Groups

Examine the permissions `-rwxr-x--`. For the user, `rwx` is calculated as  $4+2+1=7$ . For the group, `r-x` is calculated as  $4+0+1=5$ , and for other users, `--` is represented with `0`. Putting these three together, the numeric representation of those permissions is `750`.

This calculation can also be performed in the opposite direction. Look at the permissions `640`. For the user permissions, `6` represents read (4) and write (2), which displays as `rw-`. For the group part, `4` only includes read (4) and displays as `r--`. The `0` for other provides no permissions (`---`) and the final set of symbolic permissions for this file is `-rw-r-----`.

# Users and Groups

## Examples

- Remove read and write permission for group and other on **file1**:

```
[student@desktopX ~]$ chmod go-rw file1
```

- Add execute permission for everyone on **file2**:

```
[student@desktopX ~]$ chmod a+x file2
```

- Set read, write, and execute permission for user, read, and execute for group, and no permission for other on **samplendir**:

```
[student@desktopX ~]$ chmod 750 samplendir
```



# Users and Groups



## Note

The **chmod** command supports the **-R** option to recursively set permissions on the files in an entire directory tree. When using the **-R** option, it can be useful to set permissions symbolically using the **X** flag. This will allow the execute (search) permission to be set on directories so that their contents can be accessed, without changing permissions on most files. But be cautious. If a file has any execute permission set, **X** will set the specified execute permission on that file as well. For example, the following command will recursively set read and write access on **demodir** and all its children for their group owner, but will only apply group execute permissions to directories and files which already have execute set for user, group, and/or other.

```
[student@desktopX ~]# chmod -R g+rwX demodir
```

## Changing file/directory user or group ownership

A newly created file is owned by the user who creates the file. By default, the new file has a group ownership which is the primary group of the user creating the file. Since Red Hat Enterprise Linux uses user private groups, this group is often a group with only that user as a member. To grant access based on group membership, the owner or the group of a file may need to be changed.

File ownership can be changed with the **chown** command (change owner). For example, to grant ownership of the file **foofile** to **student**, the following command could be used:

```
[root@desktopX ~]# chown student foofile
```

**chown** can be used with the **-R** option to recursively change the ownership of an entire directory tree. The following command would grant ownership of **foodir** and all files and subdirectories within it to **student**:

# Users and Groups

```
[root@desktopX ~]# chown -R student foodir
```

The **chown** command can also be used to change group ownership of a file by preceding the group name with a colon (:). For example, the following command will change the group **foodir** to **admins**:

```
[root@desktopX ~]# chown :admins foodir
```

The **chown** command can also be used to change both owner and group at the same time by using the syntax **owner:group**. For example, to change the ownership of **foodir** to **visitor** and the group to **guests**, use:

```
[root@desktopX ~]# chown visitor:guests foodir
```

Only **root** can change the ownership of a file. Group ownership, however, can be set by **root** or the file's owner. **root** can grant ownership to any group, while non-**root** users can grant ownership only to groups they belong to.

## What is a process?

A *process* is a running instance of a launched, executable program. A process consists of:

- an address space of allocated memory,
- security properties including ownership credentials and privileges,
- one or more execution threads of program code, and
- the process state.

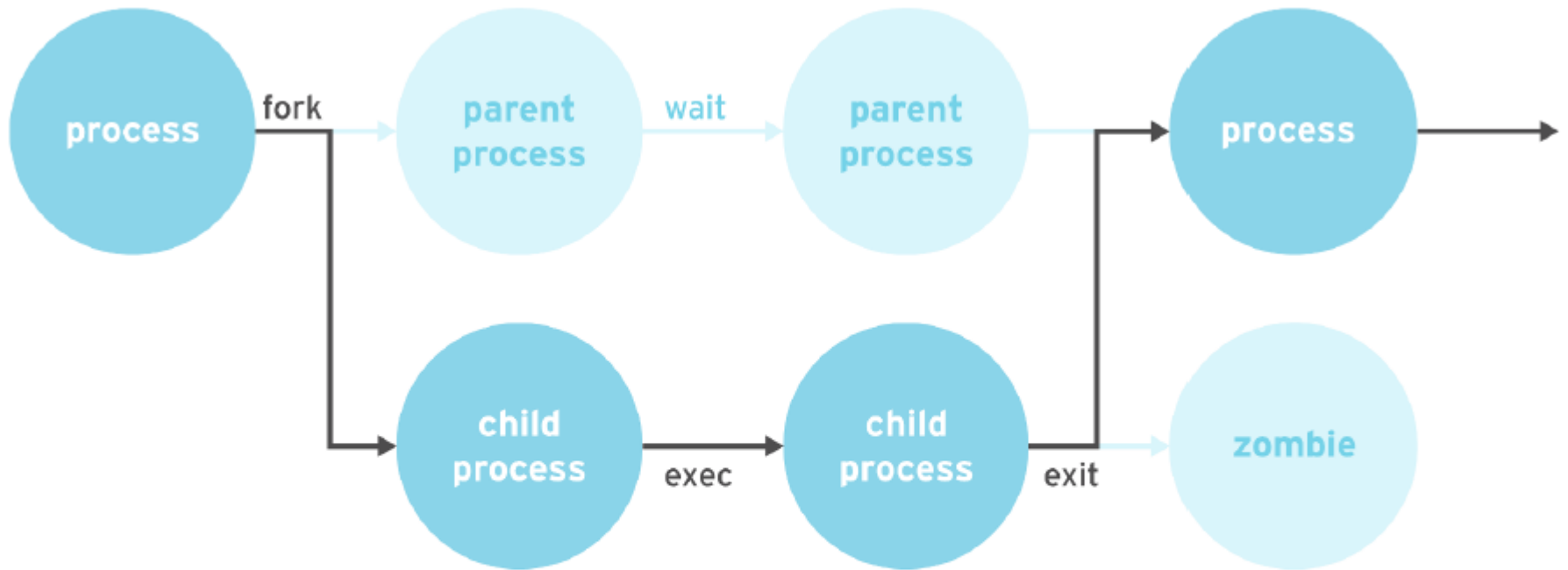
The *environment* of a process includes:

- local and global variables,
- a current scheduling context, and
- allocated system resources, such as file descriptors and network ports.

An existing (*parent*) process duplicates its own address space (**fork**) to create a new (*child*) process structure.

- Every new process is assigned a unique *process ID* (PID) for tracking and security.
- The PID and the *parent's process ID* (PPID) are elements of the new process environment.
- Any process may create a child process.
- All processes are descendants of the first system process, which is **systemd(1)** on a Linux 7 system.

# Processes



*Figure 7.1: Process life cycle*

Through the **fork** routine, a child process inherits security identities, previous and current file descriptors, port and resource privileges, environment variables, and program code. A child

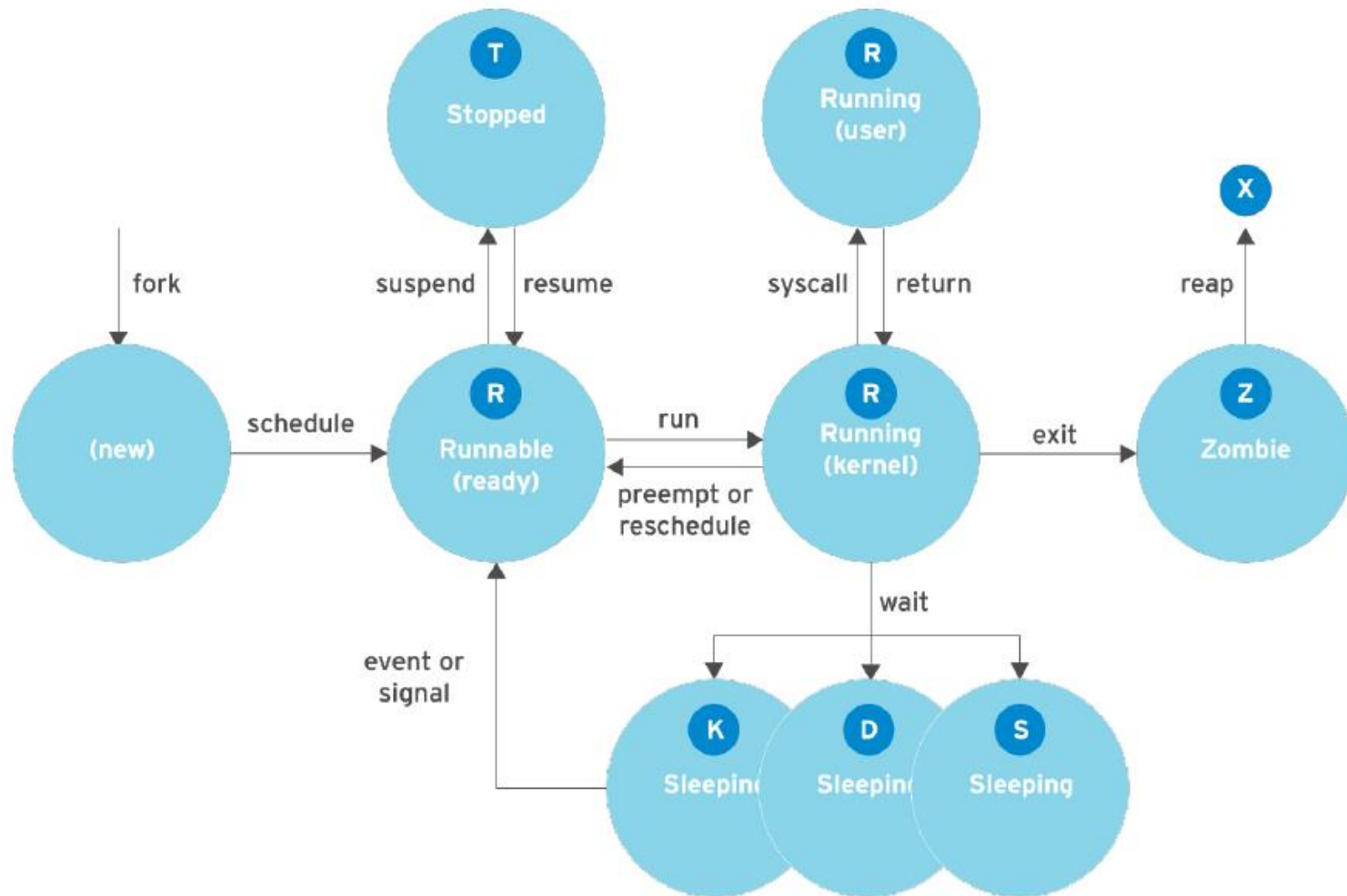
- A child process may then exec its own program code. Normally, a parent process sleeps while the child process runs, setting a request (wait) to be signaled when the child completes.
- Upon exit, the child process has already closed or discarded its resources and environment; the remainder is referred to as a zombie.
- The parent, signaled awake when the child exited, cleans the remaining structure, then continues with its own program code execution.

## Process states

- In a multitasking operating system, each CPU (or CPU core) can be working on one process at a single point in time.
- As a process runs, its immediate requirements for CPU time and resource allocation change.
- Processes are assigned a state, which changes as circumstances require.



# Processes: Linux Process States



# Processes

## Linux process states

| Name     | Flag     | Kernel-defined state name and description                                                                                                                                                                                                 |
|----------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Running  | <b>R</b> | TASK_RUNNING: The process is either executing on a CPU or waiting to run. Process can be executing user routines or kernel routines (system calls), or be queued and ready when in the <i>Running</i> (or <i>Runnable</i> ) state.        |
| Sleeping | <b>S</b> | TASK_INTERRUPTIBLE: The process is waiting for some condition: a hardware request, system resource access, or signal. When an event or signal satisfies the condition, the process returns to <i>Running</i> .                            |
|          | <b>D</b> | TASK_UNINTERRUPTIBLE: This process is also <i>Sleeping</i> , but unlike <b>S</b> state, will not respond to delivered signals. Used only under specific conditions in which process interruption may cause an unpredictable device state. |
|          | <b>K</b> | TASK_KILLABLE. Identical to the uninterruptible <b>D</b> state, but modified to allow the waiting task to respond to a signal to be killed (exited completely). Utilities frequently display <i>Killable</i> processes as <b>D</b> state. |
| Stopped  | <b>T</b> | TASK_STOPPED: The process has been <i>Stopped</i> (suspended), usually by being signaled by a user or another process. The process can be continued (resumed) by another signal to return to <i>Running</i> .                             |
|          | <b>T</b> | TASK_TRACED: A process that is being debugged is also temporarily <i>Stopped</i> and shares the same <b>T</b> state flag.                                                                                                                 |
| Zombie   | <b>Z</b> | EXIT_ZOMBIE: A child process signals its parent as it exits. All resources except for the process identity (PID) are released.                                                                                                            |
|          | <b>X</b> | EXIT_DEAD: When the parent cleans up ( <i>reaps</i> ) the remaining child process structure, the process is now released completely. This state will never be observed in process-listing utilities.                                      |

## Listing processes

The `ps` command is used for listing current processes. The command can provide detailed process information, including:

- the user identification (UID) which determines process privileges,
- the unique process identification (PID),
- the CPU and real time already expended,
- how much memory the process has allocated in various locations,
- the location of process STDOUT, known as the controllingterminal,
- the current process state.



## Important

The Linux version of **ps** supports three option formats, including:

- UNIX (POSIX) options, which may be grouped and must be preceded by a dash,
- BSD options, which may be grouped and must not be used with a dash, and
- GNU long options, which are preceded by two dashes.

For example, **ps -aux** is not the same as **ps aux**.

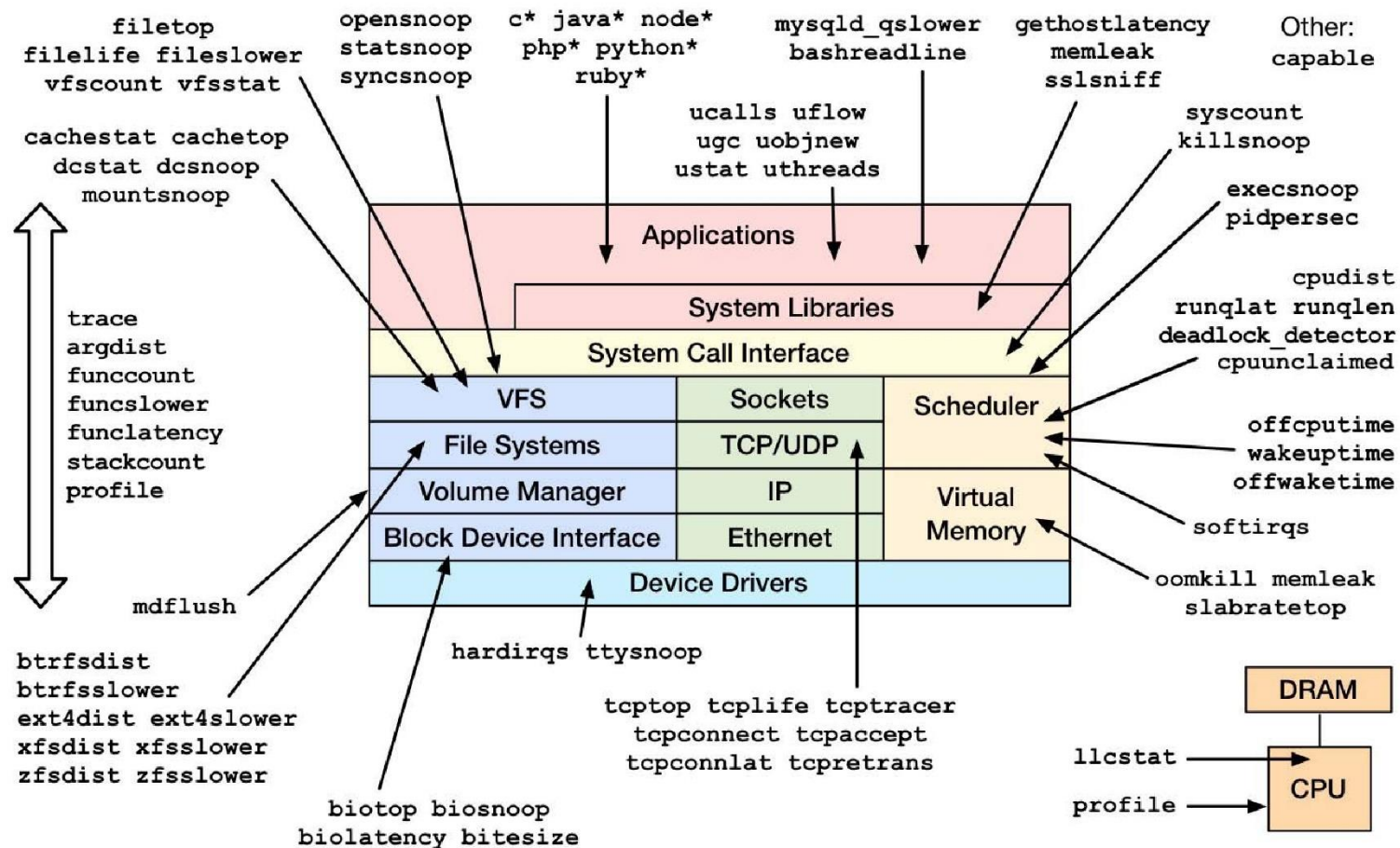
A common display listing (options **aux**) displays all processes, with columns in which users will be interested, and includes processes without a controlling terminal. A long listing (options **lax**) provides more technical detail, but may display faster by avoiding the username lookup. The similar UNIX syntax uses the options **-ef** to display all processes.

```
[student@serverX ~]$ ps aux
```

| USER | PID | %CPU | %MEM | VSZ   | RSS  | TTY | STAT | START | TIME | COMMAND               |
|------|-----|------|------|-------|------|-----|------|-------|------|-----------------------|
| root | 1   | 0.1  | 0.1  | 51648 | 7504 | ?   | Ss   | 17:45 | 0:03 | /usr/lib/systemd/syst |
| root | 2   | 0.0  | 0.0  | 0     | 0    | ?   | S    | 17:45 | 0:00 | [kthreadd]            |

# Linux bcc/BPF Tracing Tools

## Linux bcc/BPF Tracing Tools

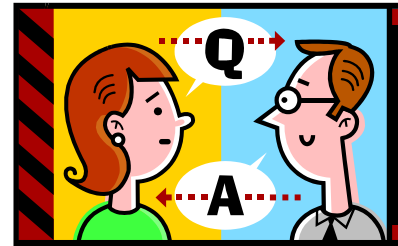


<https://github.com/iovisor/bcc#tools> 2017

# Brief break (if on schedule)

---

## Q&A?



Prof. Mehdi Pirahandeh  
E-mail : mehdi@inha.ac.kr